

COMS W4156 Advanced Software Engineering (ASE)

September 19, 2024

On this day ...



Agenda

1. Looking for Group
2. Continuous Integration
3. CI demo: Rishabh Srivastava
4. Next



Team Project Milestones

Team Formation due **tomorrow**, September 20, 11:59pm

Team Project Proposal due one week later September 27

Ideally 4 students per team, 5 students ok. Make sure:

- Everyone on the team submits the same thing
- Everyone fluent in team's choice of Java or C++
- Everyone has same or compatible development platform
- Everyone has compatible schedules



Is Anyone Still Looking For A Team?

If anyone does not yet have a team, or has a team with less than four or more than five members, please speak up NOW



Breakout Rooms If Needed

1. C++
2. Java on Mac
3. Java on Windows
4. Java on Linux
5. and up - pick an empty room and meet with your team, or stay in the main room



Agenda

1. Looking for Group
2. Continuous Integration
3. CI demo: Rishabh Srivastava
4. Next



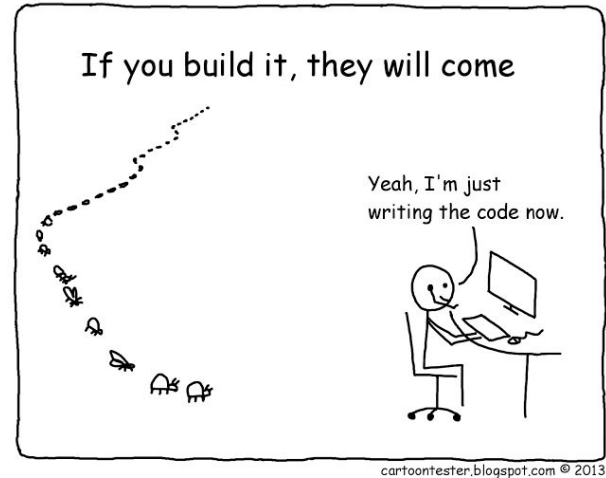
Finding Bugs - Not How, But When?

Software has bugs. This is normal.

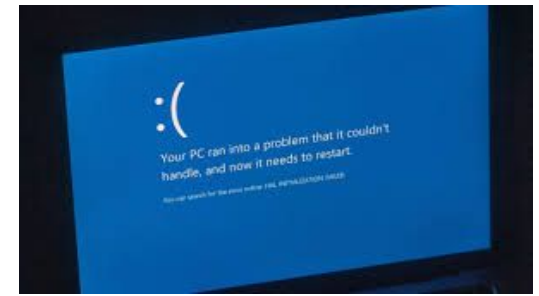
Buggy code does not matter much if it never leaves the developer's machine

Buggy code committed to shared repositories will increase time and effort for *other* software engineers on the same team,

but unlikely to be humongously expensive if bugs never make it to production



A Bug That Made It To Production



How One Bad CrowdStrike Update Crashed the World's Computers

A defective CrowdStrike update sent computers around the globe into a reboot death spiral, taking down air travel, hospitals, banks, and more with it.



Find Bugs Before Changes Leave Developer's Machine

Examine the code:
Static Analysis



If the entire system builds and runs locally, and all the style checking and static analysis tools run locally, and the entire test suite can run locally, and the developer has unlimited time and patience, ideally the developer would complete examine/execute locally,

Execute the code:
Dynamic Analysis



→ and only after that issue code review and pull requests

Synonyms for Unlikely

improbable

implausible

doubtful

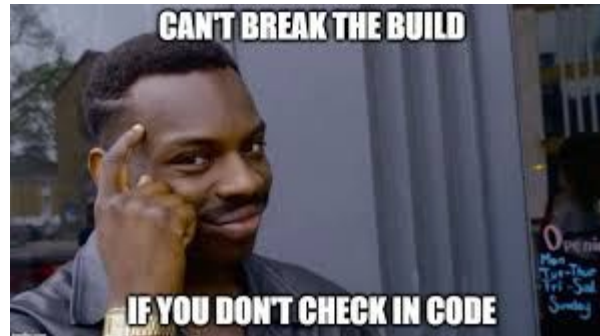
unbelievable

incredible

Don't Break The Build

Breaking the build = Errors detected during CI were traced to *your* commit

If all developers had fully tested their changes locally (on their own machine) prior to commit, and fixed any errors prior to commit, *in theory* CI should not find any new errors



In Theory vs. In Practice

Developers test their own changes locally with only a subset of the full test suite

Often just the unit tests for the changed units

Maybe integration tests across multiple changed units

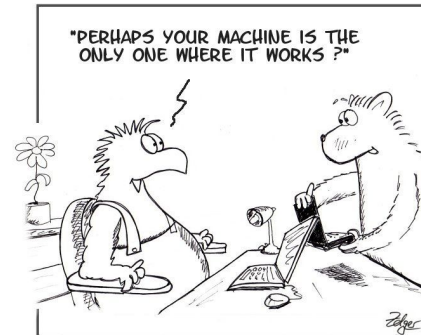
Developer machines typically do not have the resources to run the system's full regression test suite

And developers do not have the time to wait for full builds in a test environment

Theory: 

Practice: 

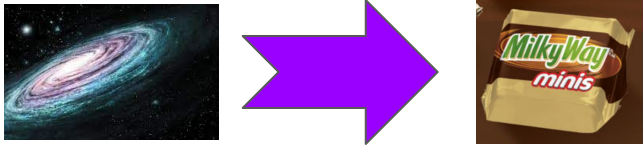
Machine Learning : 



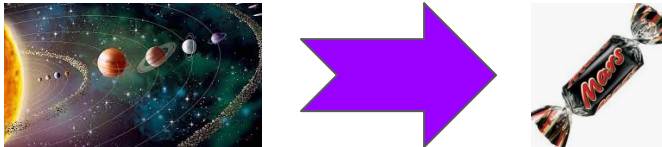
It works on my machine

Find *Some* Bugs Before Changes Leave Developer's Machine

Examine the code: Static Analysis



Execute the code: Dynamic Analysis



Style check and quick static analyses of changed files

Unit and integration tests exercise changed files

Use [git pre-commit hook](#) to automate



Find Bugs Before Changes Affect Users

Check everything.

If any code changes, check everything.

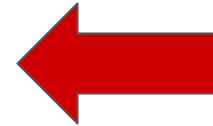
If any test changes, check everything.

If any dependency changes, check everything.

If any external data changes, check everything.

If any environment changes, check everything.

If any configuration file changes, check everything.



...

...

...



NEVER FORGETS



SOMETIMES FORGETS

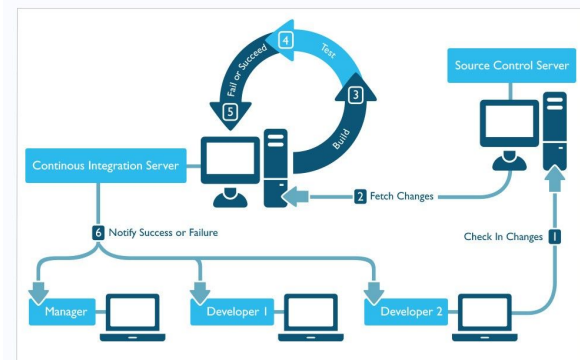


ALWAYS FORGETS

How Do We Automate All Those (Re)Checks *At Scale*?

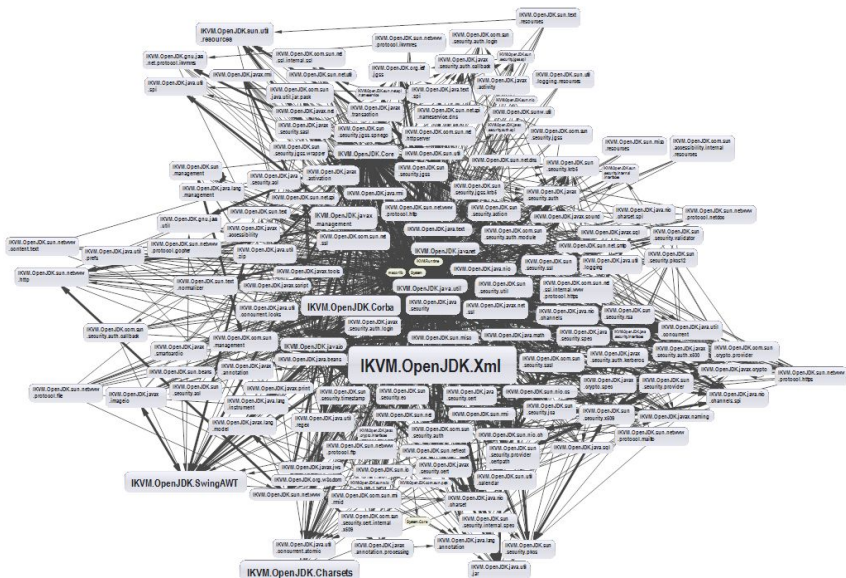
"[Continuous Integration](#)" (CI) tools hook into a version control system like github and can automatically re-build and re-check (on designated CI server) after every commit to the shared repository. Goal is to detect errors as soon as they are introduced to the shared repo

Typically full CI runs on a timer ([nightly build](#)), not literally every commit



Github provides its own built-in mechanisms supporting CI (and other capabilities) via [github actions](#) for small-scale projects. Alternatives like [CircleCI](#) also compatible with github

"I Thought Build Meant Compile"



"Build" often refers to gathering all [dependencies](#), followed by compile and link (compiled languages) or packaging together all the dependencies (non-compiled languages)

In CI context, “build” also runs analysis and testing tools, normally much more expensive than re-gathering dependencies and/or re-compiling

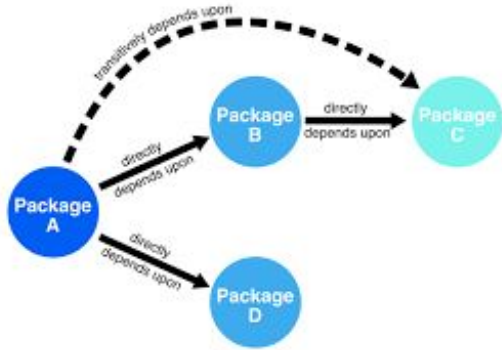
Test Selection

A common choice for commit-time testing is to employ a “[test minimization](#)” strategy that selects only new/modified test cases plus those existing test cases whose results (pass or fail) could be affected by the committed code changes

This may be a relatively small subset of all the tests, but there’s a **cost** to accurately computing the affected tests

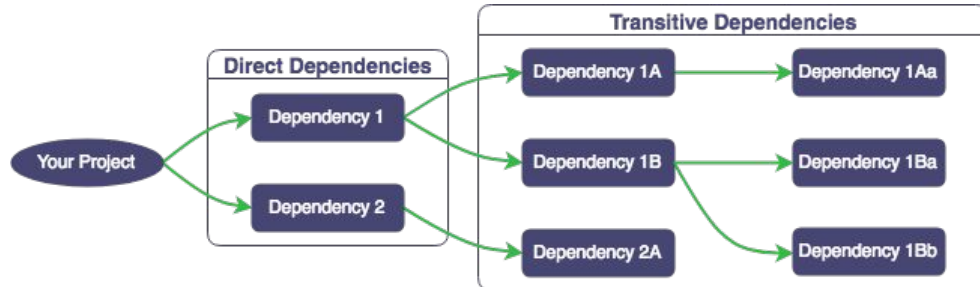


Change Analysis



Track all code c exercised by each test t during *previous* testing

If the commit changes c , or changes any other code that c depends on directly or indirectly, then re-run t



Regression Testing

Regression testing is the typical choice for nightly builds

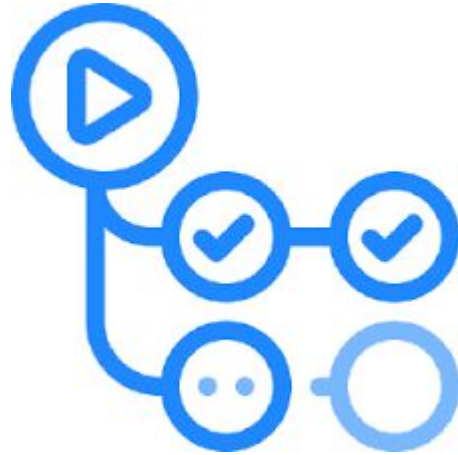
A *regression* is when fixing a bug or adding a new feature causes previously working code to break, or “regress”, sometimes code that seems completely unrelated in another part of the codebase



To guarantee that you have detected all regressions (that can be detected), you need to re-run *all* test cases that previously passed - and also all tests that previously failed, in case they now fail in different ways or (unexpectedly) pass. The whole test suite!

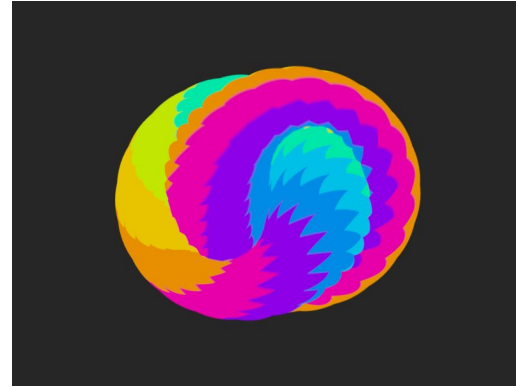
Agenda

1. Looking for Group
2. Continuous Integration
3. CI demo: Rishabh Srivastava
4. Next



CI demo: Rishabh Srivastava

[demo slides](#)



Agenda

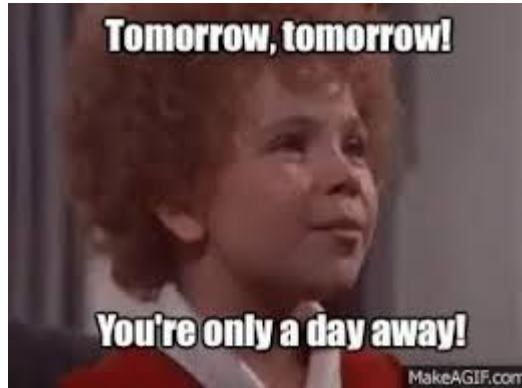
1. Looking for Group
2. Continuous Integration
3. CI demo: Rishabh Srivastava
4. **Next**



Remaining Miniproject Deadlines

[Part Individual Miniproject part 2](#) due **tomorrow** September 20 (students who registered for class September 9 or later should submit [part 1](#) together with part 2)

[Part Individual Miniproject part 3](#) due the following Friday September 27



Team Project Milestones

Team Formation **also** due tomorrow, September 20, 11:59pm

Team Project Proposal due one week later September 27

Ideally 4 students per team, 5 students ok. Make sure:

- Everyone on the team submits the same thing
- Everyone fluent in team's choice of Java or C++
- Everyone has same or compatible development platform
- Everyone has compatible schedules



Your Project Must Target an "Acceptable User Group"

- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Maybe others with societal benefit - ask me *in advance*



Your Team Project Should Build A Useful Service That Other Developers Would Want To Use



Your service should be usable remotely by lots of different clients developed by arbitrary developers, teams, organizations with similar or unrelated target users

Your service's functionality and code implementation, particularly your API(s), should be documented sufficiently that arbitrary developers, teams, organizations could plausibly build on your service

Your own team will build a sample client as part of the second iteration

EVERYTHING ABOUT DISTRIBUTED SYSTEMS

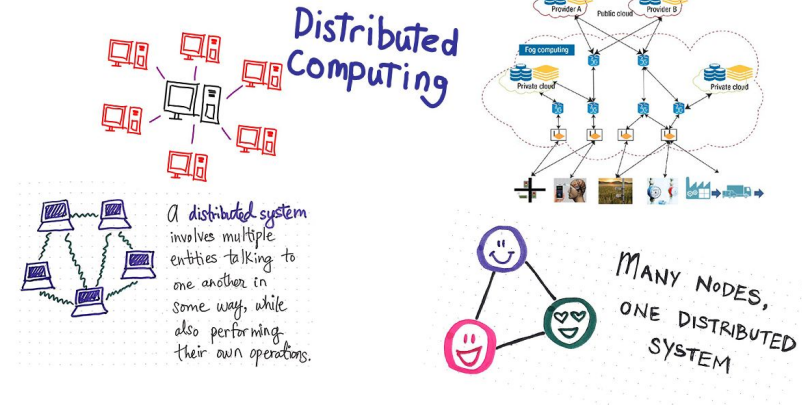
Next Week

Finish anything we didn't get to today

GCP deployment demo

Team Project Proposals

Some of you asked "what is a distributed system?" and "how is it different from cloud computing?" We will cover RPC and other kinds of APIs used in distributed systems, where the services implementing the APIs should not care whether they run in the cloud (amazon, google or microsoft manages the data center) or on traditional distributed computing infrastructure (your organization manages its own data center). Since you probably don't have your own data center, we will give you credits on GCP [Here is link to 4113](#) (distributed computing) [Here is link to 4153](#) (cloud computing)





Ask Me Anything In Scope

